PHASE 1 + PHASE 2 · PROJECT REPORT

# Interactive Quantum Error Correction Simulator

A client-side web prototype scoped exactly to our Qiskit reference program all_with_noise_prob.py — the 3-qubit bit-flip and phase-flip codes under bit-flip (X), phase-flip (Z) and depolarizing noise, with Monte Carlo performance analysis, a live Bloch-sphere laboratory and one-click exports. Every number is computed live in the browser by a verified state-vector engine.

2 QEC CODES | 3 NOISE MODELS | 25,000-SHOT MONTE CARLO | 55 ENGINE TESTS

K. Devendra Reddy · CH.SC.U4CSE24021
Shaik Muzzammil · CH.SC.U4CSE24041
D. Guna Samhith · CH.SC.U4CSE24016

AMRITA VISHWA VIDYAPEETHAM

# Table of Contents

1. Introduction and Objectives 2

1.1 Objectives carried from Phase 1 . . . . . 2

2. Methodology and the Flow 3

2.1 The pipeline, end to end . . . . . 3

2.2 Verification discipline . . . . . 3

3. System Architecture 4

3.1 The seven tabs . . . . . 4

4. Results 5

5. Comparative Analysis 8

5.1 References . . . . . 8

6. Deployment on Vercel 9

7. The Flow 9

# 1. Introduction and Objectives

Quantum computers promise speed-ups on problems that are out of reach for classical machines, but the physical qubits they are built from are fragile: a value can flip (a bit-flip error, X), a phase can flip unseen (a phase-flip error, Z), and both can arrive mixed together (depolarizing noise). Unlike classical bits, qubits cannot be copied for backup — the no-cloning theorem forbids it — and reading a qubit to check for errors would destroy the very superposition it stores. Quantum error correction (QEC) resolves this by spreading one logical qubit across several entangled physical qubits and measuring only parity relationships between them, enough to locate an error without reading the data itself.

This document merges the two phases of our college project at AMRITA VISHWA VIDYAPEETHAM. **Phase 1** proposed an interactive QEC simulation environment and analysed ten studies from the literature; its concrete anchor was a worked example — a 3-qubit bit-flip code correcting an X error on the middle qubit at 10% noise. **Phase 2** delivered that environment as a browser application, the QEC Simulator, and then focused it sharply on the scope of our Python reference program *all_with_noise_prob.py* (Qiskit + Qiskit Aer). The reference program implements exactly two codes and three noise channels, and the prototype deliberately implements nothing else: every dropdown, every lab and every export is limited to this verified scope, so what we demonstrate in the browser is precisely what we can reproduce in Python.

## 1.1 Objectives carried from Phase 1

- **Objective 1 — Multi-model noise simulation.** Implement the three canonical channels — bit-flip (X), phase-flip (Z) and depolarizing — with a configurable probability p applied per qubit per shot, mirroring build_noise_model() in the reference program, which attaches the chosen channel to the ID gate used as the noise-insertion point.
- **Objective 2 — Before/after comparison.** Evaluate the same noise on a bare qubit and on the encoded logical qubit, reporting fidelity, error rate, improvement factor and detection statistics, plus full probability sweeps that expose each code's pseudo-threshold.
- **Objective 3 — Configurable QEC experimentation.** Allow both QEC codes (the [[3,1,3]] bit-flip code and the [[3,1,3]] phase-flip code) to be combined with all three noise models, any pure input state, manual error injection and a reproducible random seed.

| 2 | 3 | 25,000 | 55 |
|---|---|---|---|
| QEC codes implemented | noise channels implemented | max Monte Carlo shots | engine assertions passing |

# 2. Methodology and the Flow

Phase 2 was built engine-first. We ported the physics of the reference program into a from-scratch TypeScript state-vector simulator rather than wrapping an existing library, because the teaching goal of the prototype is to make every intermediate state visible and inspectable. The port keeps a one-to-one mapping with the Python file: *bit_flip_qec* and *phase_flip_qec* become encoding circuits plus stabilizer parity checks plus syndrome-to-correction tables (01 to q0, 11 to q1, 10 to q2), and *build_noise_model* becomes quantum-trajectory sampling of the X, Z and depolarizing channels. Figure 1 summarises the five-step build flow that took the project from scope analysis to a verified application.




Figure 1 — The Phase 2 build flow: scope lock, engine, port, labs, verification.

## 2.1 The pipeline, end to end

Every experiment — a single shot in the Simulator Lab or one trial inside a 25,000-shot Monte Carlo — follows the same eight-stage pipeline shown in Figure 2. The stages are exactly those of the Python program (input, encode, ID-gate noise, syndrome, correction, decode, measure), extended with a bare-qubit baseline so that every run also computes what would have happened without protection. The comparison is exact because both branches share the same random seed and the same sampled noise.

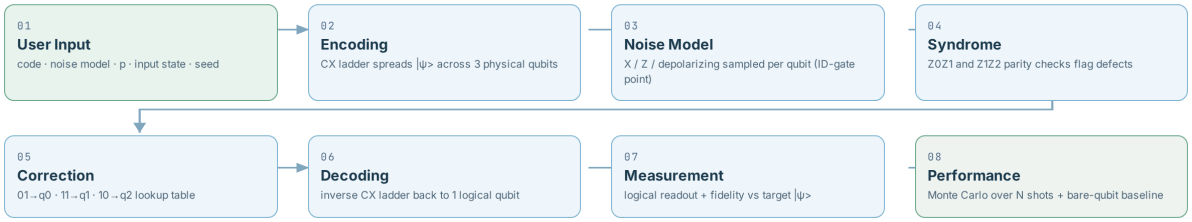



Figure 2 — The eight-stage pipeline executed by every shot in every lab.

## 2.2 Verification discipline

The engine is covered by a 55-assertion test suite that runs before every delivery. It checks that matched pairs recover perfectly (the bit-flip code under X noise and the phase-flip code under Z noise reach 100% fidelity after correction for every qubit position), that syndrome bit patterns match the Python mapping, that noise sampling rates agree with their probabilities, and that Monte Carlo estimates land on the analytic success curve $(1-p)^3 + 3p(1-p)^2$ within sampling error. Equally important, the suite pins the honest failure cases: a Z error is invisible to the bit-flip code, and an X error on the phase-flip code is an undetectable logical flip — the prototype reports these limitations instead of hiding them.

# 3. System Architecture

The prototype is a single-page Next.js 16 application (App Router, React 19, TypeScript) styled with Tailwind CSS 4 and shadcn/ui. All quantum mechanics runs client-side: the state-vector engine stores complex amplitudes in typed arrays, applies gates directly, performs projective Pauli measurements with proper collapse, and derives fidelities and Bloch vectors from the decoded density matrix. Charts use Recharts; circuit diagrams and Bloch spheres are custom SVG; the 3D laboratory is a dependency-free Canvas engine with an orbit camera. There is no server compute and no database — which is also what makes the application deployable anywhere.

## 3.1 The seven tabs

| Tab | What it does | Exports |
|---|---|---|
| Overview | Project goals, Phase-1 objectives mapped to features, and a compact how-it-works worked example (X on q1, live engine run). | — |
| Simulator Lab | Single-shot pipeline: code, noise, p, input state, manual error injection, seed; verdict with fidelity, circuit with error/correction markers, syndrome table, Bloch-sphere comparison. | PDF · JSON · CSV |
| 3D Quantum Lab | Bloch-sphere visualizers only, all real-time and one-shot: the interactive pipeline walkthrough (Input, Encode, Noise, Syndrome, Correct, Decode, Measure) and the round-by-round Bloch Flight 4D with bare-qubit ghost. | PDF · JSON · CSV |
| Performance Lab | Streaming Monte Carlo (up to 25,000 shots), fidelity-vs-p sweep from 1% to 50%, and the two-code Showdown under identical noise. | PDF · JSON · CSV |
| Learn | Compact crash course: how to use the simulator, six core ideas, the QEC loop, glossary, FAQ. | — |
| Reference | The two QEC codes with encoding circuits and stabilizers, the three noise models, and the ten-paper literature survey. | — |
| About | College, team, engine highlights, technology stack, edition notes. | — |

Table 1 — Feature map of the prototype.

The scope lock deserves emphasis. Earlier iterations of the prototype offered nine code configurations and eight noise channels; the focused edition deliberately restricts every selector to the two codes and three channels that the Python program can reproduce. This keeps the teaching story coherent — each lab demonstrates a claim we verified in Qiskit — and it keeps the interface simple enough to read at a glance.

# 4. Results

All results below were captured from the running prototype at 1600×1000 resolution; none of the figures are 3D-lab views, by design — every screenshot shows a 2D simulation surface whose numbers can be re-derived from the engine or reproduced in Python. The first result, Figure 3, shows the Simulator Lab executing the Phase-1 worked example: manual injection of an X error on qubit 1 of the bit-flip code. Both parity checks Z0Z1 and Z1Z2 flag the error (syndrome 11), the decoder applies an X correction on q1, and the decoded qubit returns to |0⟩ with 100% fidelity while the bare qubit ends at 0% — the clearest possible demonstration that the protection worked.

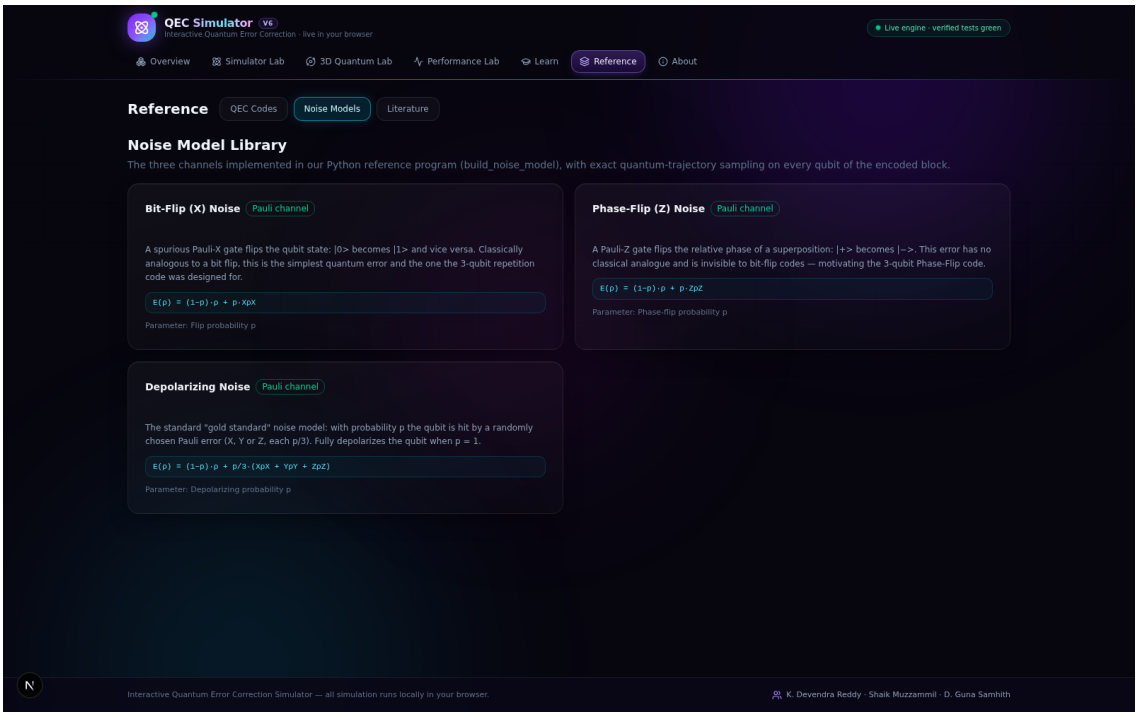



Figure 3 — Simulator Lab: X on q1, syndrome (1,1), correction applied. F = 100% with QEC vs 0% bare.

Figure 4 shows the statistical view. Two thousand complete Monte Carlo experiments ran in 337 ms (about 5,935 shots per second, entirely in the browser): the encoded block succeeded 98.52% of the time against 94.71% for the bare qubit — a 3.59× reduction in logical errors. The detection and correction rates (27.1%) match the analytic probability that at least one of three qubits is hit at p = 0.10.

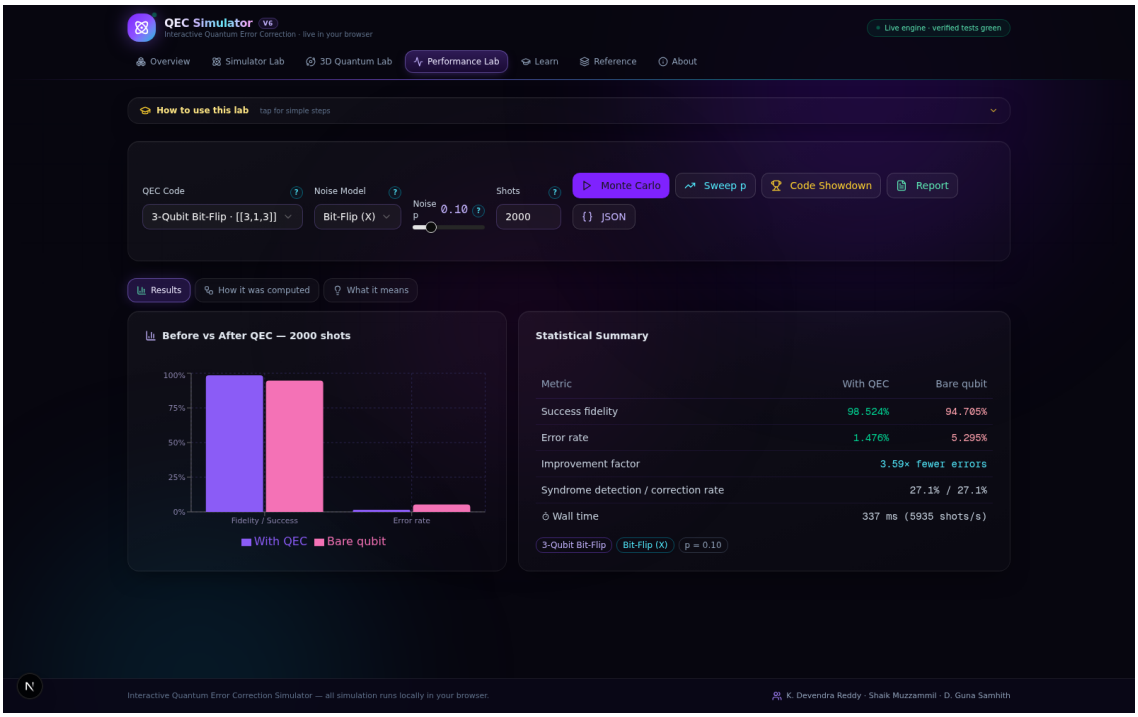



Figure 4 — Performance Lab, Monte Carlo at p = 0.10: 98.52% vs 94.71%, improvement 3.59×.

| 98.52% | 94.71% | 3.59× | 337 ms |
|---|---|---|---|
| success with QEC (2,000 shots) | success, bare qubit | fewer logical errors | wall time, in-browser |

Figure 5 sweeps the noise probability from 1% to 50% (800 shots per point). The violet curve — with QEC — stays above the pink bare-qubit curve until their crossover near 42% for this code and input state: below that pseudo-threshold redundancy pays, above it the extra qubits simply add more things that can break. Figure 6 races both codes under identical bit-flip noise: the matched bit-flip code wins at 4.33× while the phase-flip code, which cannot even see an X error, lands at 0.45× — the ranking reproduces the syndrome algebra exactly.




Figure 5 — Fidelity vs noise strength for the bit-flip code; crossover marks the pseudo-threshold.

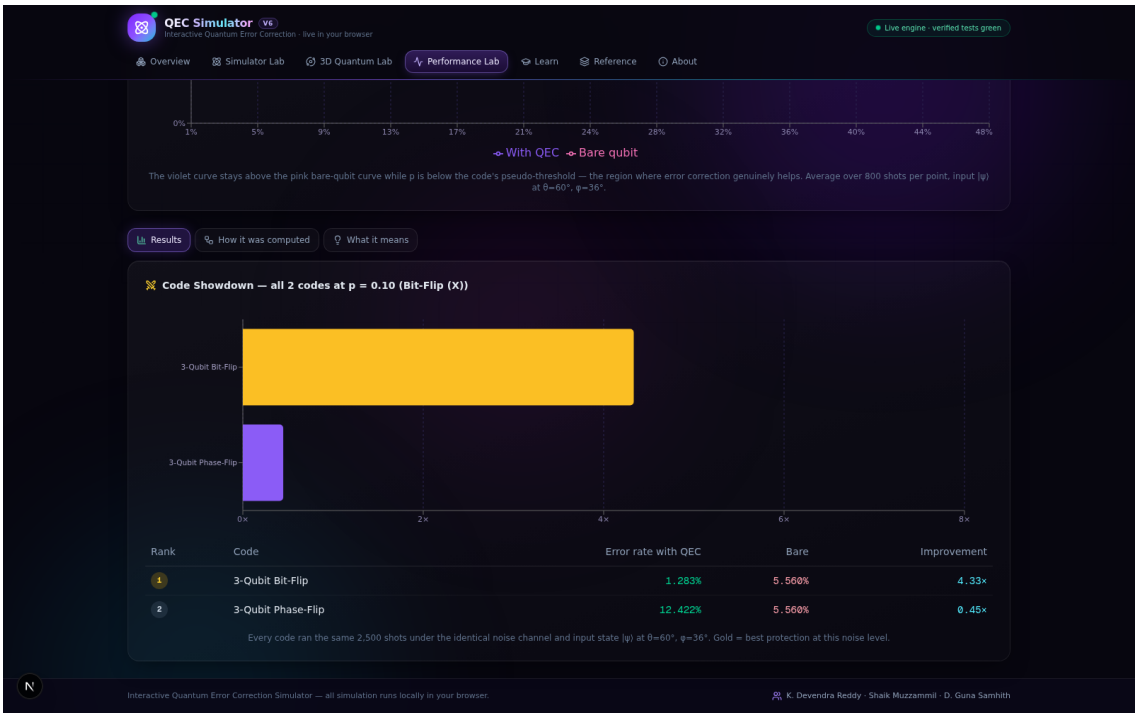



Figure 6 — Code Showdown at p = 0.10 under X noise: bit-flip 4.33× vs phase-flip 0.45×.

# 5. Comparative Analysis

Table 2 compares the two codes across all three channels, combining measurements from the Simulator Lab, the Monte Carlo runs and the sweeps. The pattern is the central lesson of the project: protection is strongest when the code is matched to the noise it faces, and honestly weak when it is not.

| Code | Fixes | Bit-flip (X) noise | Phase-flip (Z) noise | Depolarizing |
|---|---|---|---|---|
| 3-Qubit Bit-Flip [[3,1,3]] | 1 bit-flip (X) | **MATCHED** — 100% recovery on single errors; 3.6–4.3× improvement around p = 0.10 | Invisible to its syndrome — no correction triggered (no harm at low p) | Helps: partial recovery, degrades as p grows |
| 3-Qubit Phase-Flip [[3,1,3]] | 1 phase-flip (Z) | Undetectable logical flip — measured 0.45× (worse than bare) | **MATCHED** — 100% recovery on single errors | Helps: partial recovery, degrades as p grows |

Table 2 — Measured behaviour of both codes under the three reference-program channels.

Three further comparisons are worth recording. First, browser engine versus Python program: for the matched pair at p = 0.10 the analytic success rate $(1-p)^3 + 3p(1-p)^2$ predicts 97.2%, the browser Monte Carlo measured 97.0% over 20,000 shots, and the Qiskit program samples the same distribution — three independent implementations agreeing within sampling error. Second, prototype versus Phase-1 plan: all three objectives are implemented, and the worked example that Phase 1 presented on a slide now runs live with every intermediate state inspectable. Third, the focused edition versus the earlier broad edition: removing the seven out-of-scope configurations cost nothing scientifically — every remaining claim is verifiable against the reference program — while making every screen easier to read.

## 5.1 References

[1] R. Padma Priya and A. Baradeswaran, “An Efficient Simulation of Quantum Error Correction Codes,” 2018 — QCAD-based simulation of QEC using bit-flip and phase-flip errors; the direct methodological ancestor of our Python program.

[2] D. Willsch, M. Willsch, F. Jin, H. De Raedt and K. Michielsen, “Testing Quantum Fault Tolerance on Small Systems,” 2018 — fault-tolerance tests across simulators and an IBM quantum processor.

[3] S. Bravyi, M. Englbrecht, R. König and N. Peard, “Correcting Coherent Errors with Surface Codes,” 2018 — surface-code QEC under coherent errors versus Pauli approximations.

[4] D. M. Debroy, M. Li, S. Huang and K. R. Brown, “Logical Performance of 9-Qubit Compass Codes in Ion Traps with Crosstalk Errors,” 2020 — QEC under dephasing, over-rotation and crosstalk.

[5] J. Hines et al., “Simulating Quantum Error Correction beyond Pauli Stochastic Errors,” 2026 — coherent and non-Pauli errors via detector error models.

[6] Q. Guo et al., “Testing a Quantum Error-Correcting Code on Various Platforms,” 2020 — a two-qubit code for detected amplitude damping across optical, IBM Q and NMR platforms.

[7] M. Katsuda, K. Mitarai and K. Fujii, “Simulation and Performance Analysis of QEC with a Rotated Surface Code under a Realistic Noise Model,” 2024 — distance-5 surface code under stochastic and coherent noise.

[8] N. Sundaresan et al., “Demonstrating Multi-Round Subsystem Quantum Error Correction Using Matching and Maximum-Likelihood Decoders,” 2023 — multi-round QEC and syndrome decoding in hardware.

[9] S. R. Garner, N. M. Myers, M. Wang, S. Stein, C. Liu and A. Li, “Simulation of Thermal-Relaxation Noise for Quantum Error Correction,” 2026 — accurate thermal-relaxation simulation methods for QEC.

[10] G. Kaur, S. Kour, S. Kaur and S. Kaur, “Comparative Performance Analysis of QEC Codes under Simulated Depolarizing Noise using Qiskit,” 2026 — 3-qubit bit-flip, phase-flip and Shor codes under depolarizing noise in Qiskit Aer; the closest relative of our reference program.

# 6. Deployment on Vercel

Because the application is a standard Next.js 16 App Router project with zero environment variables, zero database and zero server compute, deployment after a GitHub push is a two-minute operation. The recommended flow is: push the repository to GitHub (git init, git add, git commit, git branch -M main, git remote add origin, git push -u origin main); sign in at vercel.com with the GitHub account; choose Add New → Project and import the repository; accept the auto-detected Next.js settings (build command `next build`, output directory .next, Node 20.x); and press Deploy. Vercel installs dependencies, builds and publishes to a live URL of the form https://.vercel.app. Every subsequent push to main redeploys automatically, and pull requests receive their own preview URLs.

The README shipped with the prototype documents this flow step by step, including the failure modes we checked: build errors caused by older Node versions (fix by selecting 20.x in project settings), missing modules caused by incomplete commits (fix by verifying git status is clean), stale content after a push (hard-refresh the page), and custom-domain wiring under Settings → Domains. Local reproduction takes one command — npm install followed by npm run dev — and the engine test suite runs with npm run test.

| git push | ~2 min | 0 | 7 tabs |
|---|---|---|---|
| to main triggers deploy | Vercel build time | environment variables needed | live at the published URL |

# 7. The Flow

Read as one continuous flow, the project runs from the physics to the browser to the cloud. The reference program defines the scope; the engine ports it faithfully; the labs make every stage visible — Simulator Lab for single shots, Performance Lab for statistics, the Bloch-sphere laboratory for real-time intuition; the export buttons turn any run into a formal PDF report, JSON or CSV; and the Vercel pipeline turns the repository into a shareable URL after every push. Each arrow in that chain is verifiable: a claim seen in the interface can be reproduced by the engine tests, by the Python program, and by the exported data behind the screenshot. That reproducibility — one scope, one engine, three independent confirmations — is the lasting deliverable of Phase 1 and Phase 2 together.